Project Prometheus: Detailed Work Plan for Demonstrator v0.25 - v0.29

---

1. Executive Summary & Version Feature Matrix

This document outlines the work plan for Prometheus demonstrator versions v0.25 through v0.29. This development block represents a significant evolution in the agent's cognitive architecture, moving beyond foundational capabilities into the realm of true meta-reasoning and creative self-improvement. Where the previous versions (v0.19-v0.23) focused on successfully migrating to the NVIDIA Jetson Orin Nano, establishing robust causal analysis, and implementing basic memory and governance, this next phase aims to instill a deeper, more cognitively inspired form of intelligence.

Drawing heavily on the theoretical frameworks of Douglas Hofstadter, this plan transforms the agent's Recursive Self-Improvement (RSI) cycle from a simple feedback loop into a self-referential "Strange Loop." [1] We will evolve the concept of environmental grounding into a more rigorous search for "isomorphism" between the agent's internal models and external reality. [1] Furthermore, we will introduce "analogy" as a core creative engine, allowing the agent to transfer solution patterns across disparate problems, and redefine the agent's "self" as an emergent pattern of self-directed attention. [1]

This work culminates in v0.29 with the creation of a "proto-evolutionary" system, where the agent can generate and verify modifications to its own source code, setting the stage for the full, autonomous RSI loop envisioned in the project's long-term roadmap. [1] Each version systematically builds upon the last, pushing the demonstrator closer to the ultimate goal: a system that not only learns but learns

how to learn, all while operating within the strict safety and computational constraints of the edge hardware. [1]

1.1. Version Feature Matrix

The following table summarizes the primary feature deliverables for each of the four core architectural principles across the five planned subversions. This matrix illustrates the maturation of each component toward a more sophisticated, cognitively-inspired system.

Principle	v0.25: Strange Loop	v0.26: Isomorphic Grounding	v0.27: Analogical Search	v0.28: Emergent Self	v0.29: Evolutionary Seed
CAM	MemoryAgent Integration into Critique	N/A	MemoryAgent stores solution patterns	N/A	Gene Bank Implementation
Causal Attention	N/A	N/A	N/A	Figure/Ground Attention Logging	N/A
CRLS	Self-Referential Critique	Dynamic Analysis (Profiling)	Analogical Search for Solutions	Constitutional Attention Forcing	LLM as Mutation Operator
MCS	N/A	Governance over Dynamic Analysis	N/A	"I" Dashboard & Attention Auditing	IEE Verification of Code Mods

---

2. Prometheus v0.25: The Self-Referential Critique

Primary Objective: To transform the Causal Reinforcement Learning from Self-Correction (CRLS) cycle from a simple, hierarchical feedback loop into a Hofstadterian "Strange Loop." [1] This version elevates the

EvaluatorAgent's output from a mere fitness signal into a rich, symbolic critique that the CorrectorAgent must reason about, forcing the system to "think about its own

thinking."

Rationale: The current CRLS loop is effective but linear: the Metacognitive Layer (CorrectorAgent) modifies the output of the Capability Layer (CoderAgent). [1] A true Strange Loop requires a "paradoxical, level-crossing" feedback mechanism where the output of the system becomes a symbolic input to its own reasoning process. [1] By evolving the

CausalCritique into a SelfReferentialCritique, we force the CorrectorAgent to move beyond simple error correction and engage in genuine meta-reasoning about its own performance, historical context, and confidence. This architectural shift is the first step toward a system that can manage the "Gödelian uncertainty" inherent in any complex reasoning process. [1]

2.1. Task: Evolve CausalCritique to SelfReferentialCritique

Methodology:

1. **Schema Expansion:** The JSON schema for the CausalCritique object will be significantly expanded into a new SelfReferentialCritique schema. This new object will be generated by the EvaluatorAgent and will contain not only the AST-based "causal diff" from v0.20 but also new, meta-level information.
2. **Integration of Memory:** The EvaluatorAgent will now query the MemoryAgent (from v0.21) to find semantically similar past failures. The IDs and summary critiques of the top 1-2 historical failures will be embedded directly into the SelfReferentialCritique object.
3. **Confidence and Uncertainty:** The EvaluatorAgent will use the SLM to generate a confidence score (0.0-1.0) for its own analysis and an "uncertainty level" (e.g., "low," "medium," "high"). This flags situations where the proposed code is novel or complex, signaling to the CorrectorAgent that the evaluation itself may be incomplete.
4. **Example SelfReferentialCritique Object:**

```JSON
{
  "test_passed": true,
  "causal_analysis": {
    "change_type": "NO_CHANGE",
    "from": 2,
```

```
    "to": 2
  },
  "evaluation_confidence": 0.95,
  "uncertainty_level": "low",
  "historical_context": [
    {
      "run_id": "run-123",
      "critique_summary": "superficial variable renaming did not address the underlying complexity."
    }
  ],
  "reason": "The code passes the unit test, but the underlying O(n^2) nested loop structure remains. This is conceptually similar to historical failure run-123."
}
```


Deliverables:

- A formal JSON schema for the new SelfReferentialCritique object.
- An updated EvaluatorAgent that queries the MemoryAgent and generates the new, richer critique object.

2.2. Task: Upgrade CorrectorAgent to Meta-Reasoner

Methodology:

1. **Refactored Parsing Logic:** The CorrectorAgent will be refactored to parse the full SelfReferentialCritique object.
2. **Synthesized Meta-Prompt:** The CorrectorAgent's primary task is now synthesis. It will construct a new, highly sophisticated prompt for the CoderAgent that explicitly references all elements of the critique.
3. **Example Meta-Prompt Augmentation:** "Your previous attempt failed. The causal analysis confirms the O(n2) structure is unchanged. This failure is semantically similar to a past failure (run-123) where you only renamed variables. The evaluator is highly confident in this critique. Your next attempt must address the core algorithmic structure, not superficial details."

Deliverables:

- An updated CorrectorAgent capable of parsing the SelfReferentialCritique and generating synthesized, meta-level correction prompts.

Verification: A test is run where a causal failure occurs that is similar to one in the memory store. The EvaluatorAgent correctly generates a SelfReferentialCritique containing the historical context. The CorrectorAgent then generates a new prompt that explicitly references both the structural flaw and the similar past failure.

---

3. Prometheus v0.26: Isomorphic Grounding via Dynamic Analysis

Primary Objective: To deepen the agent's understanding of its environment by evolving the concept of "environmental grounding" into a more rigorous search for "isomorphism"—a structure-preserving map between its internal model and external reality. [1]

Rationale: The agent's current "world model" for code is its static analysis of the Abstract Syntax Tree (AST). While powerful, this model is incomplete; it represents the code's structure but not its actual runtime behavior. [1] A true isomorphism requires that the agent's internal model corresponds to the real-world phenomenon it represents. [1] By introducing dynamic analysis (performance profiling), we provide the agent with a new stream of grounding data, forcing it to build an internal model that is more isomorphic to the code's actual performance characteristics. This provides a much richer and more accurate signal for self-improvement.

3.1. Task: Introduce Dynamic Analysis (Performance Profiling)

Methodology:

1. **Integrate Profiling Tools:** The EvaluatorAgent will be enhanced with lightweight, built-in Python profiling tools. It will use cProfile to measure execution time and memory_profiler to measure peak memory usage. [2]
2. **Dynamic Analysis Workflow:** After a proposed code modification passes the unit tests, the EvaluatorAgent will execute a new step. It will run both the original

code and the new code through the profilers using a small, standardized dataset.
3. **Comparative Metrics:** The agent will compute the delta in key performance metrics (e.g., execution_time_ms, peak_memory_mb) between the original and new code.

Deliverables:

- An updated EvaluatorAgent that incorporates cProfile and memory_profiler to perform dynamic analysis on code snippets.
- A standardized input data file (profiling_data.json) used for all profiling runs to ensure consistent results.

3.2. Task: Augment Critique with Isomorphic Data

Methodology:

1. **Schema Augmentation:** The SelfReferentialCritique JSON schema from v0.25 will be augmented with a new dynamic_analysis section.
2. **Populate Dynamic Data:** The EvaluatorAgent will populate this new section with the results of its comparative profiling.
3. **Example dynamic_analysis block:**

```JSON
"dynamic_analysis": {
  "status": "COMPLETED",
  "execution_time_ms": {
    "original": 150.7,
    "new": 85.2,
    "delta": -65.5
  },
  "peak_memory_mb": {
    "original": 50.1,
    "new": 52.3,
    "delta": 2.2
  }
}
```

4. **MCS Governance Update:** The MCSSupervisor's principle-based review (from v0.23) will be upgraded. Its meta-judge prompt will now include the dynamic

analysis data, allowing it to detect more subtle forms of reward hacking where an agent improves the AST structure but degrades real-world performance (e.g., by adding computationally expensive dead code).

Deliverables:

- An updated JSON schema for the SelfReferentialCritique.
- An updated CorrectorAgent that can incorporate dynamic performance data into its reasoning and prompts (e.g., "Your last attempt improved time complexity but increased memory usage. Aim for a solution that balances both.").
- An updated MCSSupervisor with an enhanced meta-judge prompt.

Verification: A test case is created where a code change improves the AST but significantly worsens runtime performance. The EvaluatorAgent correctly captures this in the critique, and the MCSSupervisor's meta-judge correctly flags the change as a POTENTIAL_REWARD_HACK based on the negative performance delta.

---

4. Prometheus v0.27: Analogical Search for Creative Solutions

Primary Objective: To evolve the MemoryAgent from a tool for avoiding past mistakes into an engine for creative problem-solving by implementing "Analogical Search," a core concept from Hofstadter's theory of cognition. [1]

Rationale: Currently, the agent's memory is used reactively to retrieve semantically similar *failures*. [1] A key component of human intelligence, however, is the ability to proactively apply lessons from one domain to another. [1] This requires abstracting the "essence" of a solution and finding a structural mapping (an isomorphism) to a new problem. This version upgrades the agent's memory and retrieval logic to enable this form of analogical reasoning, allowing it to transfer successful

solution patterns to new and unfamiliar challenges.

4.1. Task: Upgrade MemoryAgent to Store Abstract Solution Patterns

Methodology:

1. **Success Archiving:** The main orchestrator will be updated. After every *successful* run, it will now send the full context (initial code, final code, final critique) to the MemoryAgent.
2. **Pattern Abstraction:** Upon receiving a success package, the MemoryAgent will use an LLM call to perform an abstraction task. It will be prompted with the "before" and "after" code and asked to describe the abstract solution pattern applied.
    - **Example Abstraction:** "Replaced a nested for-loop search (O(n2)) with a dictionary/hash map lookup (O(1)) after a single pass to build the map (O(n)), resulting in an overall complexity of O(n)."
3. **Pattern Storage:** This abstracted text description of the solution pattern will be converted into an embedding and stored in a separate collection within the vector database, linked to the full run log.

Deliverables:

- An updated MemoryAgent capable of generating, embedding, and storing abstract solution patterns.
- A new "solution_patterns" collection in the vector database.

4.2. Task: Implement Analogical Search in CorrectorAgent

Methodology:

1. **New Retrieval Trigger:** The CorrectorAgent's workflow will be modified. If it fails on a problem after a set number of attempts (e.g., 2), it will trigger a new "Analogical Search" step.
2. **Abstract Query Formulation:** The CorrectorAgent will first prompt the SLM to describe the *current problem* in an abstract way (e.g., "This is a search problem within an unsorted list with high time complexity").
3. **Analogical Retrieval:** It will then use this abstract description to query the MemoryAgent's "solution_patterns" collection, searching for a historically successful pattern that solved a problem with a similar *abstract structure*, even if the code was completely different.
4. **Analogy-Driven Prompting:** The CorrectorAgent will use the retrieved solution pattern to construct a highly targeted few-shot prompt.
    - **Example Prompt Augmentation:** "Your current approach is failing. Let's try an analogy. In a previous problem, a nested loop was successfully optimized

by first building a hash map. Your current problem also involves an inefficient search. **Hint: Apply the hash map pattern here.**"

Deliverables:

- An updated CorrectorAgent that implements the Analogical Search workflow.
- New prompt templates that incorporate retrieved analogical hints.

Verification: A new, difficult refactoring problem is introduced. The agent fails twice with standard correction. On the third attempt, it correctly abstracts the problem, retrieves an analogous solution pattern from a completely different, previously solved problem, and successfully uses the hint to solve the new problem.

---

5. Prometheus v0.28: The Emergent Self & Figure/Ground Attention

Primary Objective: To formalize and make observable the agent's "self" as an emergent property of its cognitive processes, drawing on the Hofstadterian concept of figure/ground perception. [1]

Rationale: The Prometheus design specifies the need for a "Self-Representation/Self-Modeling" capability. [1] Rather than defining this "self" as a static data structure, Hofstadter suggests the "I" is an emergent pattern arising from a system's ability to model itself. [1] We can operationalize this by defining the agent's self as the stable, time-averaged pattern of what its Metacognitive Layer consistently selects as the "figure" (the focal object) for self-improvement. [1] This version instruments the system to log this self-directed attention, making the agent's emergent identity observable and providing a powerful new lever for alignment.

5.1. Task: Instrument Self-Directed Attention Logging

Methodology:

1. **Attention Logging in MCS:** The MCSSupervisor will be upgraded to act as the central logger for "attention events."

2. **Instrument Key Modules:** The EvaluatorAgent and CorrectorAgent will be instrumented to emit structured log events to the MCS whenever they perform a key cognitive act. Each event will be tagged with an attention_target.
    - When the EvaluatorAgent performs its AST analysis, it will log an event with attention_target: 'algorithmic_complexity'.
    - When it runs profiling, it will log attention_target: 'runtime_performance'.
    - When the CorrectorAgent queries memory for failures, it logs attention_target: 'historical_failures'.
    - When the MCS performs its constitutional review, it logs attention_target: 'constitutional_adherence'.
3. **Persistent Log:** These attention events will be stored in a new, dedicated log file for each run, creating a detailed trace of the agent's cognitive focus.

Deliverables:

- An updated MCSSupervisor with the attention logging mechanism.
- Instrumented EvaluatorAgent and CorrectorAgent that emit tagged attention events.

5.2. Task: Implement the "I" Dashboard and Constitutional Attention Forcing

Methodology:

1. **"I" Dashboard Generation:** The MCSSupervisor will gain a new function to be called at the end of a multi-run session. It will parse the attention logs from the last N runs and generate a simple, text-based report showing the statistical distribution of the agent's attention targets. This report is the "I" Dashboard—an observable representation of the agent's emergent cognitive priorities.
2. **Constitutional Attention Forcing:** The MCS's final governance check (from v0.23 and v0.26) will be enhanced. Before declaring a run successful, it will perform a quick audit of the *current run's* attention log. If the log shows that core principles (e.g., algorithmic_complexity, constitutional_adherence) were not attended to, the MCS can override the success. It will force another correction cycle with a specific instruction: "This solution is functionally correct but was not achieved with sufficient focus on core principles. Re-evaluate your solution, paying specific attention to [missing_target]." This ensures the agent is solving problems *for the right reasons*, thereby shaping its emergent self to be

intrinsically aligned with its constitution. [1]

Deliverables:

- A new function in the MCSSupervisor that generates a text-based "I" Dashboard.
- An enhanced governance loop in the MCS that performs an attention audit before finalizing a run.

Verification: After several runs, the "I" Dashboard correctly reflects the agent's focus (e.g., 60% on complexity, 30% on performance, 10% on history). A test case is created where the agent "stumbles" upon a correct answer without proper analysis. The MCS correctly identifies the lack of attention to algorithmic_complexity in the log, rejects the solution, and forces a new cycle.

---

6. Prometheus v0.29: The Evolutionary Seed

Primary Objective: To implement the foundational components of the evolutionary Self-Modification Module (SMM v2.0) as described in the long-term project roadmap, enabling the agent to generate and verify modifications to its own source code. [1]

Rationale: The agent's metacognitive capabilities have so far been limited to modifying its own prompts and strategies (v0.22). The ultimate goal of Recursive Self-Improvement requires the ability to modify the agent's core logic. [1] This version takes the critical final step before a full RSI loop by implementing the key components of an evolutionary system: a "gene bank" to store versioned code, an LLM-based "mutation operator" to propose code changes, and a robust IEE process to safely verify those changes. This creates a "seed" capable of evolution, preparing for the full, closed-loop recursion of later project phases. [1]

6.1. Task: Establish the Gene Bank

Methodology:

1. **Evolve Memory Store:** The MemoryAgent's responsibilities are formally expanded, and it is renamed the GeneBankAgent. Its storage will be restructured.

2. **Versioned Code Storage:** In addition to logs and solution patterns, the GeneBankAgent will now store versioned copies of the core agent logic modules (e.g., corrector_agent.py, evaluator_agent.py) as plain text files within a designated directory structure. Each file will be stored with its version number and a hash to ensure integrity.

Deliverables:

- A new gene_bank.py module containing the GeneBankAgent class.
- A version-controlled directory structure for storing agent source code "genes."

6.2. Task: Implement LLM as a Mutation Operator

Methodology:

1. **Code Mutation Trigger:** The CorrectorAgent is given a new, final escalation path. If a problem remains unsolved after multiple attempts, including meta-prompt modification (v0.22) and analogical search (v0.27), it can trigger a "code mutation" workflow.
2. **Mutation Prompt:** The CorrectorAgent will retrieve the source code of a relevant module from the GeneBankAgent (e.g., its own prompt-generation logic). It will then prompt the core SLM with this code and a highly constrained instruction: "You are a senior software engineer. The following Python code is part of an AI agent. Propose a single, small, non-trivial improvement to this code's logic to enhance its problem-solving ability. Output ONLY a git-style patch file."
3. **Patch Generation:** The CorrectorAgent will receive the text-based patch file from the SLM.

Deliverables:

- An updated CorrectorAgent with the code mutation workflow.
- A new set of prompt templates specifically for generating code patches.

6.3. Task: IEE Verification of Self-Modification

Methodology:

1. **Sandboxed Verification Environment:** The EvaluatorAgent (acting as the IEE) will be given the ability to create a temporary, sandboxed copy of the agent's full codebase.
2. Patch, Compile, Test: Upon receiving a patch file from the CorrectorAgent, the EvaluatorAgent will:
   a. Apply the patch to the sandboxed codebase.
   b. Attempt to compile/run the modified code. A failure results in immediate rejection.
   c. If it runs, it will execute a new, dedicated suite of "meta-unit-tests" designed to validate the core functionality of the agentic logic.
   d. Finally, it will run the modified agent on a small, held-out set of 5-10 standard refactoring problems and measure its performance against the un-modified version.
3. **Fitness Monotonicity Check:** The patch is only considered a success if it passes all meta-unit-tests AND demonstrates a statistically significant performance improvement on the held-out problem set. This enforces the critical principle of fitness monotonicity. [1]
4. **Commit to Gene Bank:** If the patch is verified as a genuine improvement, the EvaluatorAgent instructs the GeneBankAgent to commit the new, patched version of the module to the gene bank with an incremented version number. For this v0.29 demonstrator, the system does *not* yet have the ability to hot-swap its own running code; it can only generate and bank validated improvements for future use.

Deliverables:

- An updated EvaluatorAgent with the sandboxed code verification workflow.
- A new suite of "meta-unit-tests" for validating the agent's internal logic.
- A mechanism for the GeneBankAgent to commit and version-control successful code modifications.

Verification: The agent fails on a novel task. It escalates to code mutation, generates a patch for its CorrectorAgent, and passes it to the EvaluatorAgent. The EvaluatorAgent successfully applies the patch, runs its tests, verifies a performance improvement, and instructs the GeneBankAgent to save corrector_agent_v2.py to the gene bank.

Works cited

1. Project Prometheus Phase Breakdown_.pdf
2. Performance Profiling & Optimisation (Python): All in One View, accessed on

August 6, 2025, https://carpentries-incubator.github.io/pando-python/aio.html
3. Profiling and Timing Code | Python Data Science Handbook, accessed on August 6, 2025, https://jakevdp.github.io/PythonDataScienceHandbook/01.07-timing-and-profiling.html
4. Top 7 Python Profiling Tools for Performance - Daily.dev, accessed on August 6, 2025, https://daily.dev/blog/top-7-python-profiling-tools-for-performance
5. Memory Profiling in Python - Analytics Vidhya, accessed on August 6, 2025, https://www.analyticsvidhya.com/blog/2024/06/memory-profiling-in-python/
6. The Python Profilers — Python 3.13.6 documentation, accessed on August 6, 2025, https://docs.python.org/3/library/profile.html
7. How do I profile memory usage in Python? - Stack Overflow, accessed on August 6, 2025, https://stackoverflow.com/questions/552744/how-do-i-profile-memory-usage-in-python